

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:		Reg. No.:	
Date:		Duration:	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

- 1. Problem Analysis**
 - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.
- 2. Project Structure Design**
 - Design and explain the folder structure of your project repository.
 - Justify the purpose of each major folder and file included in the repository.
- 3. Git Repository Initialization**
 - Create a Git repository for the project.
 - Explain the steps involved in repository initialization and describe the purpose of the essential Git files.
- 4. Git Branching Strategy**
 - Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
 - Justify why the selected branching model is appropriate for the assigned project.
- 5. Version Control Workflow**
 - Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
 - Explain the purpose of each Git operation performed.

6. **Commit History Analysis**
 - Present the commit history of the project.
 - Explain how commit messages follow good version control practices and contribute to project maintenance.
7. **Docker Environment Design**
 - Explain why Docker is suitable for the assigned project.
 - Identify the application components that require containerization.
8. **Dockerfile Development**
 - Design and explain the Dockerfile required to containerize the application.
 - Describe the purpose of each instruction used in the Dockerfile.
9. **Docker Compose Design**
 - Develop a Docker Compose configuration for the project.
 - Explain the services, networking, volumes, environment variables, and dependencies used.
10. **Container Deployment and Testing**
 - Demonstrate the execution of the application using Docker containers.
 - Explain the testing procedure and provide evidence that the application runs successfully.
11. **Benefits of Git and Docker**
 - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
12. **Challenges and Improvements**
 - Identify the challenges encountered during implementation.
 - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository &	Repository initialized	Repository and	Basic Git setup	Incorrect or incomplete Git

Branching Strategy(20%)	correctly with appropriate branching strategy, clear workflow, and proper repository organization.	branching strategy are mostly correct.	with limited branching implementation.	repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

Intelligent AI-Based Smart Hotel Operations and Guest Experience Management System

CO3-ASSESSMENT 1

Course:

Software Engineering / Code Review and Version Control Evaluation

Name:

Davis Damien M

192521124

Problem Analysis

Problem statement

Hotels manage reservations, room allocation, housekeeping, maintenance, guest requests, billing, and customer support through multiple disconnected systems. Manual coordination causes delays, overbooking, poor guest experiences, inefficient staff utilization, and limited operational insight. An AI-based smart hotel operations and guest experience management system integrates these functions into a single intelligent platform that automates operations and personalizes guest services using machine learning and analytics.

Project objectives

- Automate hotel operational workflows.
- Provide AI-driven guest recommendations and personalized services.
- Predict room occupancy and optimize resource allocation.
- Enable real-time housekeeping and maintenance management.
- Improve guest satisfaction through a mobile/web interface.
- Generate analytics dashboards for hotel administrators.

Functional requirements

- User authentication (guest, staff, admin).
- Room booking and availability management.
- AI recommendation engine (room upgrades, dining, amenities).
- Housekeeping task assignment and tracking.
- Maintenance request management.
- Billing and payment processing.
- Notifications (email/SMS/app).
- Admin dashboard with reports and occupancy analytics.

Expected outcomes

- Reduced operational workload.
- Faster guest request handling.
- Improved occupancy forecasting.
- Higher guest satisfaction.

- Scalable and portable deployment using Docker.
- Collaborative development using Git.

2. Project Structure Design

Repository folder structure

```
hotel-ai-system/  
├── backend/  
│   ├── app/  
│   ├── models/  
│   ├── routes/  
│   ├── services/  
│   ├── ai/  
│   ├── requirements.txt  
│   └── Dockerfile  
├── frontend/  
│   ├── src/  
│   ├── public/  
│   ├── package.json  
│   └── Dockerfile  
├── database/  
│   └── init.sql  
├── docker-compose.yml  
├── .env  
├── .gitignore  
├── README.md  
├── docs/  
│   ├── architecture.png  
│   ├── workflow.png  
│   └── report.pdf  
└── tests/  
    ├── backend/  
    └── frontend/
```

Purpose of major folders and files

- backend/: API, authentication, business logic, AI services.
- frontend/: Guest and admin web interface.

- database/: Database initialization scripts.
- docs/: Architecture diagrams and documentation.
- tests/: Automated unit and integration tests.
- docker-compose.yml: Multi-container orchestration.
- .env: Environment configuration.
- .gitignore: Excludes unnecessary files from version control.
- README.md: Project overview and setup instructions.

3. Git Repository Initialization

Steps

```
git init
git add .
git commit -m "Initial project structure"
git branch -M main
git remote add origin https://github.com/username/hotel-ai-system.git
git push -u origin main
```

Essential Git files

- .git/: Internal repository metadata.
- .gitignore: Prevents committing build files, virtual environments, logs, and secrets.
- README.md: Documents project purpose and usage.

4. Git Branching Strategy

Branch model

```
main
├── develop
│   ├── feature-auth
│   ├── feature-booking
│   ├── feature-ai-recommendation
│   ├── feature-housekeeping
│   └── feature-dashboard
```

└─ release/v1.0
└─ hotfix/login-bug

Justification

- main: Stable production-ready code.
- develop: Integration branch for ongoing development.
- feature/*: Isolated implementation of individual modules.
- release/*: Final testing before production deployment.
- hotfix/*: Urgent production bug fixes.

This model supports parallel team development while protecting production stability.

5. Version Control Workflow

Example workflow

```
git checkout -b feature-booking  
git add backend/routes/booking.py  
git commit -m "Add room booking API"
```

```
git checkout develop  
git merge feature-booking
```

```
git checkout -b feature-ai-recommendation  
git add backend/ai/recommendation.py  
git commit -m "Implement AI room recommendation service"
```

```
git checkout develop  
git merge feature-ai-recommendation
```

```
git checkout main  
git merge develop
```

```
git pull origin main  
git push origin main
```

Conflict resolution

```
git status  
git add resolved_file.py  
git commit -m "Resolve merge conflict in booking service"
```

Purpose:

- Branch creation: Isolates development work.
- Commit: Saves logical changes.
- Merge: Integrates completed features.
- Conflict resolution: Reconciles concurrent modifications.
- Pull/push: Synchronizes local and remote repositories.

6. Commit History Analysis

Example commit history

a1b2c3d Initial project structure
 b2c3d4 Add authentication module
 c3d4e5 Implement room booking API
 d4e5f6 Integrate PostgreSQL database
 e5f6g7 Add AI recommendation engine
 f6g7h8 Implement housekeeping management
 g7h8i9 Create admin dashboard
 h8i9j0 Add Docker configuration
 i9j0k1 Resolve booking merge conflict
 j0k1l2 Prepare release v1.0

Good commit practices

- Uses imperative mood (Add, Implement, Integrate).
- One logical change per commit.
- Clear and descriptive messages.
- Facilitates debugging and code review.
- Improves project maintainability and traceability.

7. Docker Environment Design

Why Docker is suitable

- Consistent development and production environments.
- Eliminates dependency conflicts.
- Simplifies deployment.
- Supports microservice architecture.

- Enables rapid scaling.

Components requiring containerization

- Backend API service.
- Frontend web application.
- PostgreSQL database.
- Redis cache (optional).
- AI inference service.

8. Dockerfile Development

Backend Dockerfile

FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "app.main", "--host", "0.0.0.0", "--port", "8000"]

Instruction explanation

- FROM: Base Python image.
- WORKDIR: Sets working directory.
- COPY: Copies dependency and source files.
- RUN: Installs Python packages.
- EXPOSE: Documents application port.
- CMD: Starts the FastAPI server.

9. Docker Compose Design

docker-compose.yml

version: '3.9'

```
services:
  backend:
    build: ./backend
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: postgres://hotel@db:5432/hoteldb
    depends_on:
      - db

  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    depends_on:
      - backend

  db:
    image: postgres:15
    environment:
      POSTGRES_USER: hotel
      POSTGRES_PASSWORD: password
      POSTGRES_DB: hoteldb
    volumes:
      - db_data:/var/lib/postgresql/data

volumes:
  db_data:
```

Explanation

- backend: REST API service.
- frontend: Web interface.
- db: PostgreSQL database.
- depends_on: Ensures startup order.
- volumes: Persistent database storage.
- environment: Database configuration.
- Networking: Docker Compose creates a shared bridge network.

10. Container Deployment and Testing

Deployment

`docker-compose build`

`docker-compose up -d`

Verification

`docker ps`

Expected containers:

- `hotel-backend`
- `hotel-frontend`
- `hotel-db`

Testing procedure

- Open <http://localhost:3000>.
- Log in as admin.
- Create a room booking.
- Verify booking appears in the dashboard.
- Check backend API:
`curl http://localhost:8000/api/health`
- Inspect database records using PostgreSQL client.

Expected result: All services communicate successfully and booking operations complete without errors.

11. Benefits of Git and Docker

Git benefits

- Parallel team collaboration.
- Complete version history.
- Easy rollback.
- Branch-based feature development.
- Code review through pull requests.
- Traceability of changes.

Docker benefits

- Environment consistency.
- Portable deployment across systems.
- Faster onboarding.
- Simplified dependency management.
- Horizontal scalability.
- Easier maintenance and updates.

Combined, Git and Docker provide reliable CI/CD-ready development and deployment workflows.

12. Challenges and Improvements

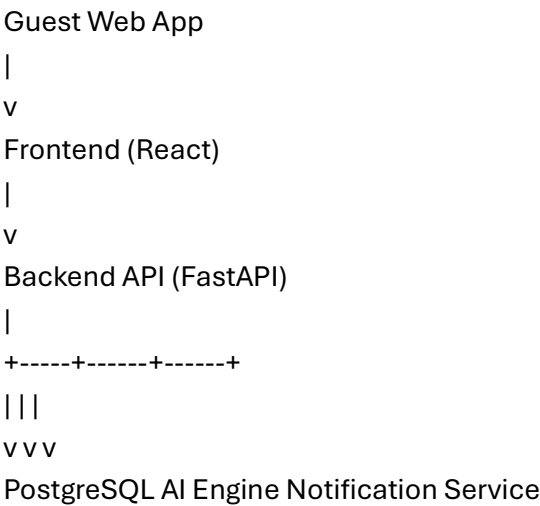
Challenges encountered

- Merge conflicts during concurrent development.
- Docker networking configuration.
- Environment variable management.
- Database initialization timing.
- Large Docker image sizes.

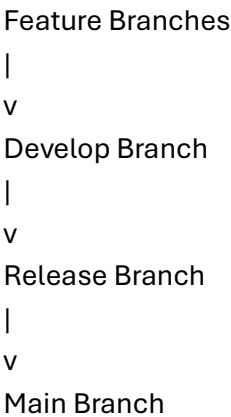
Improvements

- Use GitHub Flow with protected branches.
- Add pull request templates and code reviews.
- Integrate GitHub Actions CI/CD.
- Use Docker multi-stage builds.
- Add Nginx reverse proxy.
- Implement Kubernetes deployment.
- Add Redis caching.
- Integrate monitoring with Prometheus and Grafana.
- Implement automated backup and disaster recovery.

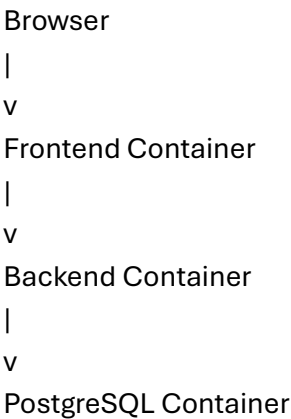
Architecture Diagram



Git Workflow Diagram



Docker Deployment Diagram



Conclusion

The Intelligent AI-Based Smart Hotel Operations and Guest Experience Management System demonstrates how Git enables structured collaborative software development while Docker provides portable and consistent deployment across environments. The project architecture supports AI-powered hotel automation, scalable microservice deployment, and maintainable version-controlled development suitable for modern hospitality management systems.